

FINLEDGER · ACCOUNTS PAYABLE AUTOMATION

Person 2 — Platform / Intake / Data

FinLedger AP · India · Tally-first · September 2026 Seat: tenancy, RLS, object store, unique link, inbound email, ingest, virus/hash, documents table, Markdown file prep.

Product (read this first)

FinLedger is a per-client Accounts Payable machine for Indian CAs and SMBs. Vendors send bills to one official link or one official email. The platform extracts the invoice, scores risk, maps ledgers with client-scoped RAG, waits for a human when the bill is risky, then posts an approved `VoucherDraft` into that client's ERP. Tally Prime first.

What it does. Takes unstructured vendor PDFs/photos and turns them into posted purchase vouchers with open bills that age. GST / TDS / MSME native. Not a GST filer, not a bank, not a card issuer.

How a bill moves. Vendor sends → Person 2 stores markdown/images → Person 1 extracts `CanonicalInvoice` and maps a `VoucherDraft` via RAG → Person 3 scores and reviews → Person 4 posts to Tally. Extraction never posts. The mapping LLM never emits Tally XML.

Full story. docs/PRODUCT.md and docs/HOW_IT_WORKS.md.

Mission

Give every client two official doors on day one. Land every file in one documents table. Store raw bytes under `client_id`. Produce markdown + page images so Person 1 can extract. Never become a second inbox. Never mix clients.

One-line path Person 2 owns:

link | email | CSV → auth + virus + hash + store → documents row + extract job

Layers owned

#	Layer	Person 2 piece
1	Channels	Unique link, inbound email, vendor portal stub later, CSV. WhatsApp is phase 9.
2	Ingest	Auth token, MIME parse, virus/hash, split attachments, per-client object store, documents queue.
15	Tenancy	Every row and path is <code>client_id</code> . RLS or schema-per-firm.

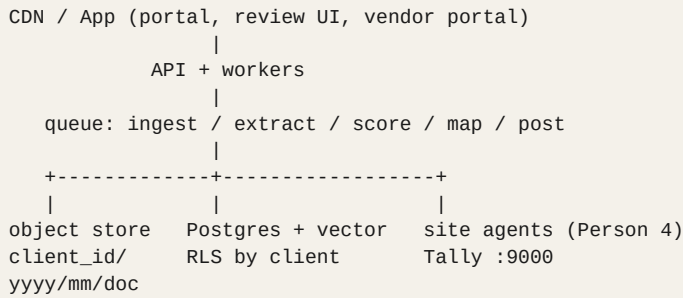
Not owned: prompts, risk math, review UX, Tally XML.

System context (doors)

Door	Address	Rules
Unique link	https://app.../i/{client_public_token}	Rotatable token, rate-limit, virus scan. PDF/JPG/PNG/ZIP. Optional PO + note.
Inbound email	invoices.{slug}@inbound.domain	One invoice per email preferred. First attachment PDF. SPF/DKIM. Phish flag if sender ≠ vendor domain.
Vendor portal	GSTIN login (later)	Same documents table.
CSV / ERP pull	Onboard + migration	Bulk bills; Person 4 may pull open payables on connect.
WhatsApp	Dedicated Business number later	Same queue. Personal chat is not an official door.

Vendor instruction: send only to this link or this email. Anything else is not received.

Runtime architecture



Suggested workers Person 2 owns: ingest_worker only. Extract/score/map/post workers are other seats, but Person 2 owns the queue names and the documents row lifecycle until extracted.

Data model Person 2 creates

clients

```
id, name, erp_type, gstins[], public_token, inbound_slug, auto_post_cap, created_at
```

documents

id, client_id, channel (link|email|csv|portal|whatsapp), source_meta (from, subject, ip), object_path, content_hash, mime, page_count, status, virus_ok, phish_flag, timestamps

document_assets

document_id, kind (raw|markdown|page_image|layout_json), path, page_no

ingest_jobs

```
id, document_id, state, attempts, last_error
```

Every select/insert goes through RLS on `client_id`. Object path is always `client_id/yyyy/mm/doc_id/...`

Ingest pipeline

1. Auth the token or verify inbound email signature. 2. MIME parse. Prefer one invoice per message. 3. Virus scan + content hash. Duplicate hash on same client + vendor later is Person 3; hash is stored now. 4. Split ZIP / multi-attach into child documents if needed. 5. Write raw file to object store. 6. Digital PDF → Markdown (pdfplumber / pdfminer). Scanned PDF / photo → render pages to images. Mixed → digital text first, OCR regions later (Person 1). 7. Write raw_markdown, layout_blocks, page images. 8. Enqueue extract with document_id.

Excel vendor statements: MarkItDown XLSX, treat as statement not a single bill. Flag document_class=statement.

Security

- Rotatable public token on the upload link.
- Rate-limit per token and per IP.
- SPF/DKIM on inbound. Flag sender domain $\neq$ vendor domain (Person 3 uses this mark).
- No personal Gmail / personal WhatsApp as a valid door.
- ERP credentials are Person 4's table, still keyed by `client_id`. Person 2 provides the tenancy pattern.
- Vector / RAG tables that Person 1 writes must live in the same RLS world. Person 2 reviews the schema.

Interfaces

To Person 1

```
document_id, client_id, raw_markdown, layout_blocks, page_image_paths, source_channel
```

To Person 3

- documents list API filtered by `client_id` + status
- signed URL for raw PDF in the review UI
- client workspace + policy columns (Person 3 defines flags, Person 2 stores them)

To Person 4

- place to persist `erp_credentials` (encrypted) and `mapping_table`
- place to persist masters snapshots for RAG refresh

From all

- every row includes `client_id`. Person 2 rejects schemas that omit it.

Week plan (Person 2)

Week	Ship
0	Client workspace, RLS, object store, unique link, inbound email, documents table
1	MarkItDown + page images + extract job publish
2	Masters write path. CSV stub.
3	Queue reliability, retries, virus, poison-queue
4	Ageing data pull storage from Person 4

Acceptance tests

- Two clients cannot read each other’s files, rows, or embeddings.
- Upload via link creates a documents row and an extract job.
- Email with one PDF attachment creates the same shape as the link.
- ZIP of two PDFs becomes two documents.
- Virus-positive file never reaches extract.
- Object path contains `client_id`.
- Re-upload of the same bytes stores the same hash so Person 3 can duplicate-check.

Refusals for this seat

- Second inbox per channel
- Accepting owner personal WhatsApp / Gmail as official intake
- Global bucket without `client_id` prefix
- Sending raw pixels to Person 1 without attempting MarkItDown on digital PDFs
- Blocking the Tally loop on vendor portal or WhatsApp